NVIDIA DGX Spark

ARM SIMD命令の
オセロ思考ルーチンへの活用

NEON / SVE2 によるビットボード高速化技法

C++ サンプルコード + SIMD命令解説 + API仕様付き

2026年4月

1. はじめに

オセロ（リバーシ）は8×8の盤面上で黒と白の石を交互に置き、相手の石を挟んで裏返すゲームです。コンピュータオセロでは、いかに高速に大量の局面を探索できるかが強さに直結します。

NVIDIA DGX Sparkは、GB10 Grace Blackwell Superchipを搭載したデスクトップAIワークステーションです。CPUにはARMv9.2アーキテクチャのCortex-X925（10コア）とCortex-A725（10コア）を採用しており、NEON（128ビット固定長SIMD）とSVE2（128ビットスケーラブルベクトル拡張）の2つのSIMD命令セットをサポートしています。

本資料では、これらのSIMD命令がオセロの思考ルーチンの各処理にどのように活用できるかを、図・C++サンプルコード・SIMD命令解説・API仕様を交えて詳しく解説します。

2. ビットボードとSIMDレジスタの対応

オセロの盤面は8×8＝64マスです。各マスの状態を1ビットで表現すれば、盤面全体をちょうど64ビット整数1つで表せます。黒石の配置を表す64ビットと白石の配置を表す64ビット、合計128ビットで盤面全体を保持するこの手法を「ビットボード」と呼びます。

DGX SparkのNEON/SVE2レジスタはちょうど**128ビット幅**です。つまり、黒と白のビットボードをSIMDレジスタ1本にぴったり格納できます。

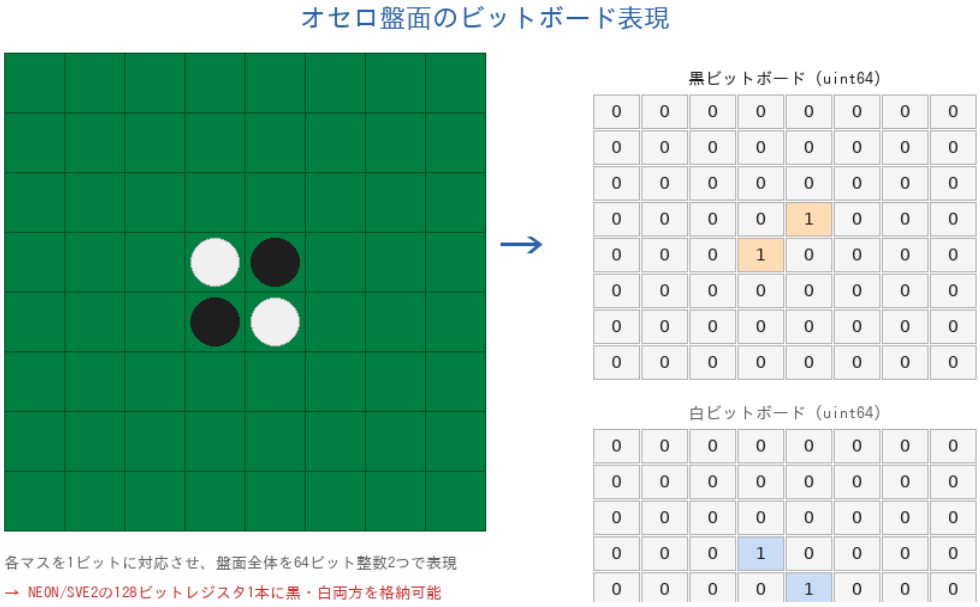


図1: オセロ盤面のビットボード表現とSIMDレジスタへの格納

コード例: ビットボードのNEONレジスタ操作

以下のコードでは、黒白2つの64ビットボードをNEONの128ビットレジスタ（uint64x2_t）に格納し、ロード・ストアする基本操作を示します。

```
#include <arm_neon.h>

struct Board {
    uint64_t black;   // 黒石ビットボード
    uint64_t white;   // 白石ビットボード
};

/*
 * load_board(b)
 *   概要 : Board構造体の黒白ビットボードを
 *          NEONの128bitレジスタ1本にまとめてロードする
 *   b    : const Board& — ロード元の盤面構造体
 *            b.black → レーン0 (下位64bit)
 *            b.white → レーン1 (上位64bit)
 *   戻値 : uint64x2_t  — 黒白を格納した128bitSIMDレジスタ
 */
inline uint64x2_t load_board(const Board& b) {
```

```
    uint64_t data[2] = { b.black, b.white };
    /*
     * vld1q_u64(ptr)
     *   ptr : const uint64_t* — ロード元アドレス
     *         ptr[0](黒) → レーン0, ptr[1](白) → レーン1
     *   戻値: uint64x2_t     — 128bitSIMDレジスタ
     */
    return vld1q_u64(data);
}

/*
 * store_board(v)
 *   概要 : NEONの128bitレジスタからBoard構造体へ書き戻す
 *   v    : uint64x2_t  — 書き戻すSIMDレジスタ
 *            レーン0 (下位64bit) → b.black
 *            レーン1 (上位64bit) → b.white
 *   戻値 : Board — 復元された盤面構造体
 */
inline Board store_board(uint64x2_t v) {
    Board b;
    uint64_t data[2];
    /*
     * vst1q_u64(ptr, val)
     *   ptr : uint64_t*   — 書き込み先アドレス
     *   val : uint64x2_t  — ストアするSIMDレジスタ
     *         レーン0 → ptr[0](黒), レーン1 → ptr[1](白)
     *   戻値: void
     */
    vst1q_u64(data, v);
    b.black = data[0];
    b.white = data[1];
    return b;
}

/*
 * get_empty(b)
 *   概要 : 盤面上の空きマス(黒でも白でもないマス)を
 *          ビットマップとして取得する
 *   b    : const Board& — 判定対象の盤面構造体
 *   戻値 : uint64_t     — 空きマスのビットマップ
 *            1のビット位置 = 空きマス
 */
inline uint64_t get_empty(const Board& b) {
    uint64x2_t bw = load_board(b);
    /*
     * vorrq_u64(a, b)
     *   a    : uint64x2_t — 第1オペランド(黒ビットボード)
     *   b    : uint64x2_t — 第2オペランド(白ビットボード)
     *   戻値 : uint64x2_t — a | b の各レーンビットOR結果
     */
    uint64x2_t occupied = vorrq_u64(
        vreinterpretq_u64_u32(
            /*
             * vmvnq_u32(a)
             *   a    : uint32x4_t — 入力ベクトル
             *   戻値 : uint32x4_t — ~a 全ビットNOT結果
             *   ※ 黒|白 を反転 → 空きマスのビットが1になる
             */
            vmvnq_u32(vreinterpretq_u32_u64(bw))),
        vreinterpretq_u64_u32(
            vmvnq_u32(vreinterpretq_u32_u64(bw))));
    return vgetq_lane_u64(occupied, 0);
```

```
}
```

リスト1: NEON intrinsicを用いたビットボードの128ビットレジスタ操作

◆ SIMD命令 詳細解説

[1] ― NEON LD1命令。メモリから128ビット(64bit×2要素)を1サイクルで1本のSIMDレジスタへロード。黒石(data[0])がレーン0、白石(data[1])がレーン1に格納されます。

[2] ― NEON ST1命令。SIMDレジスタの128ビットをメモリへ1命令で書き戻します。ロードの逆操作です。

[3] ― NEON ORR命令(Quadword/64bit lanes)。2本の64bitレーンを同時にビットOR。黒∪白のビットマップを1命令で求めます。

[4] ― NEON MVN命令(Quadword)。全ビットをビット反転(NOT)。ORR結果にNOTをかけることで「空きマス」ビットマップが1命令で完成します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vld1q_u64	uint64x2_t	const uint64_t *ptr	ptrが指すメモリから64bit×2要素(128bit)を連続ロー ptr[0]→レーン0、ptr[1]→レーン1に格納。アライメ 不要。
vst1q_u64	void	uint64_t *ptr, uint64x2_t val	valの128bitをptrへ連続ストア。レーン0→ptr[0]、レ 1→ptr[1]。vld1q_u64の逆操作。
vorrq_u64	uint64x2_t	uint64x2_t a, uint64x2_t b	a、bの各対応レーンをビットOR。黒石∪白石の論理 を2レーン同時に計算する。
vmvnq_u32	uint32x4_t	uint32x4_t a	aの全ビットをビット反転(NOT)。32bitレーン×4と 処理するが実質128bit全反転。空きマスのNOT取得 用。
vget_low_u64	uint64_t	uint64x2_t a	128bitベクトルのレーン0(下位64bit)をスカラーとし 取得。
vget_high_u64	uint64_t	uint64x2_t a	128bitベクトルのレーン1(上位64bit)をスカラーとし 取得。

3. 合法手生成の8方向同時探索

オセロで石を置けるかどうかを判定するには、ある位置から8方向（上下左右＋斜め4方向）に対して「相手の石が連続し、その先に自分の石がある」かを確認する必要があります。

3.1 スカラー処理の問題点

従来のスカラーコードでは、8方向それぞれについてループを回す必要がありました。1方向あたり最大6回のシフト＋比較が必要なため、合法手判定だけで多数の命令を消費します。

3.2 SIMDによる並列化

SVE2の128ビットレジスタを8つの16ビットレーンに分割し、各レーンに1方向の探索状態を割り当てます。1つのSIMDシフト命令で8方向すべてを同時にシフトし、1つのAND命令で8方向すべての挟み判定を同時に行えます。

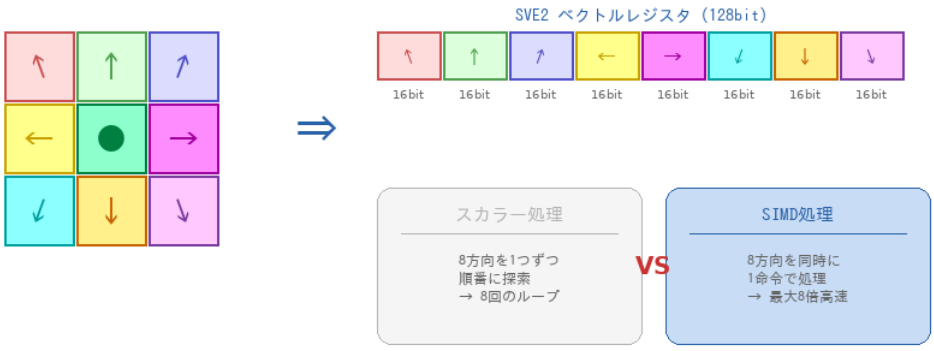


図2: 8方向探索のSIMD並列化の概念

3.3 SIMDの「レーン」とは

SIMDレジスタは、1本の物理レジスタ内を等幅の区画に分割して使います。この区画1つ1つを「レーン」と呼びます。DGX Sparkの128ビットレジスタの場合、64ビット×2レーン、32ビット×4レーン、16ビット×8レーン、8ビット×16レーンのいずれかに分割できます。

1つのSIMD命令を発行すると、全レーンで同一の演算が同時に実行されます。例えば128ビットレジスタを64ビット×2レーンに分割すれば、1命令のシフトで「右方向のシフト」と「左方向のシフト」を同時に処理できます。レーン数を増やすほど並列度が上がりますが、1レーンあたりのビット幅は狭くなります。



図2-補足1: SIMD レーンの概念 ― 128 ビットレジスタの分割パターン

リスト2では64ビット×2レーンを使い、1回のSVE2演算で2方向を同時に処理しています。8方向を2方向ずつ4ペアに分けることで、ループ回数を半減させています。

3.3.1 レーン幅はどう決まるのか ― 命令が指定する

ここで疑問が生じます。128ビットのレジスタは物理的には単なるビット列であり、境界線はどこにもありません。**プロセッサはどうやってレーン数を把握するのでしょうか？**

答えは、**レジスタ側ではなく、命令（オペコード）側がレーン幅を指定する**という仕組みです。同じ128ビットレジスタに対して、異なる命令を使えば異なるレーン幅で演算が行われます。

ARM NEON/SVE2のintrinsic関数名には、必ずデータ型を示すサフィックスが付いています。例えば：

svld1_u64 → 「64ビット符号なし整数」として扱え → 128÷64 = **2レーン**

svmla_s16_x → 「16ビット符号付き整数」として扱え → 128÷16 = **8レーン**

vcnt_u8 → 「8ビット符号なし整数」として扱え → 128÷8 = **16レーン**

プロセッサは命令のオペコードを見た瞬間にレーン幅を認識し、内部の演算回路を自動的に切り替えます。プログラマが意識するのは「どのサフィックスの命令を選ぶか」だけです。レジスタに対して「今から64ビットモードにする」のような設定操作は一切不要です。



図2-補足12: 命令のサフィックスがレーン幅を決定する仕組み

本資料のコードでは、合法手生成（リスト2）では _u64 サフィックスで64ビット×2レーンを使い、評価関数（リスト5）では _s16 で16ビット×8レーンを使い、popcount（リスト6）では _u8 で8ビット×16レーンを使っています。**同じ128ビットレジスタを、処理内容に最適なレーン幅で使い分けている**ことになります。

3.4 述語制御のSIMD命令とは

SVE2の命令には、ほぼすべてに「述語レジスタ」（プレディケートレジスタ、型名 svbool_t）を第1引数として渡します。述語レジスタは各レーンに対して TRUE（処理する）または FALSE（スキップする）を指定するビットマスクです。

述語制御の最大の利点は、**条件分岐（if文）を完全に排除できること**です。スカラー処理では「このレーンを処理するか？」を毎回if文で判定しますが、SVE2では述語がTRUEのレーンだけが自動的に演算され、FALSEのレーンはハードウェアレベルでスキップされます。分岐予測ミスによるパイプラインストールが一切発生しません。



図2-補足2: 述語(プレディケート)制御の仕組みとリスト2での使われ方

リスト2では svptrue_b64() で全レーンをTRUEにした述語を使っています。これは全レーンを対象に演算したいからです。一方、第5章の述語フィルタリングでは svwhilelt や svcmpne で条件に応じた述語を動的に生成し、合法手レーンだけを選択的に処理します。

SVE2命令の**_x サフィックス**（svlsr_x、svand_x 等）は、述語がFALSEのレーンの結果が「未定義」でよいことを示します。ハードウェアは未定義レーンの演算を完全にスキップできるため、**_z（ゼロ埋め）や _m（マージ）よりも高速**に実行されます。リスト2では全レーンTRUEですが、将来の拡張で部分的にFALSEになる場合も_xが最適です。

3.5 なぜシフト＋ANDで合法手が分かるのか

ここまでSVE2命令の並列化技法を解説しましたが、そもそも「自石をシフトして相手石とANDを取る」操作を繰り返すだけで、なぜ石を置ける場所（合法手）が分かるのでしょうか。この節ではアルゴリズムの原理を具体例で説明します。

オセロで石を置けるマスの条件は、**①自石のすぐ隣から相手石が1個以上連続しており、②その連続の先に空きマスがある**ことです。つまり「自石→相手石列→空きマス」のパターンが1方向でもあれば、その空きマスは合法手です。

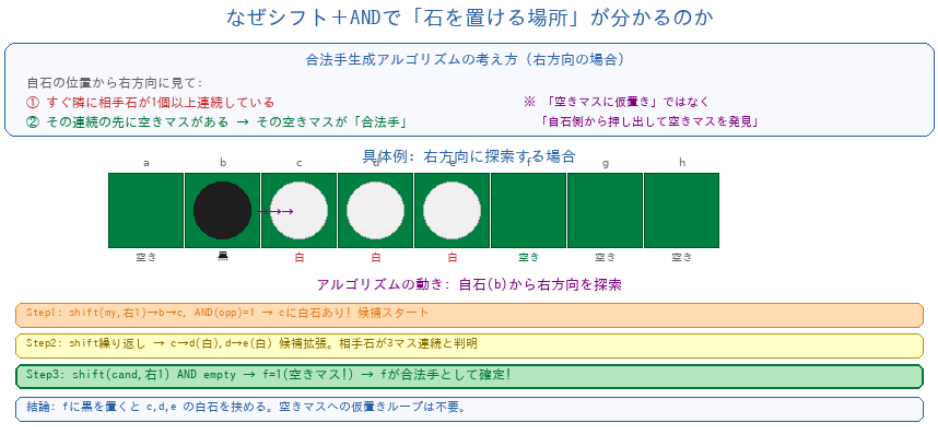


図2-補足3: 合法手生成アルゴリズムの考え方ー 自石→相手石列→空きマスを検出

ビット演算では、この条件判定を次の3つの操作に分解します。

原理①: シフト＝「隣のマスに進む」操作。ビットを1つ右シフトすることは、盤面上で1マス右に進むことと同じです。下方向なら8ビットシフト（1行＝8マス）、斜め右下なら9ビットシフトに対応します。

原理②: AND(opp)＝「進んだ先に相手石があるか」の判定。シフト後のビットと相手石マスクのANDを取ると、進んだ先に相手石があれば1、なければ0になります。1なら「挟み候補」として蓄積し、シフトを継続します。

原理③: 最後のAND(empty)＝「相手石列の先が空きマスか」の確認。相手石の連続が途切れた先のビットと空きマスマスク(empty = ~(my|opp))のANDを取ります。空きマスがあれば＝そこに石を置ける＝合法手確定。空きマスでなければ不合法です。

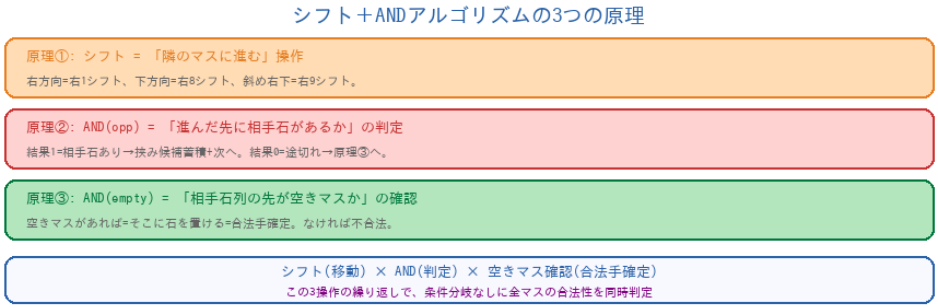


図2-補足4: シフト＋ANDアルゴリズムの3つの原理

3.5.1 成功例: 合法手が見つかる場合

具体的なビット操作を追いかけてみましょう。下図の例では、自石がb列とh列にあり、相手石がe,f,g列にある盤面を考えます。b列の黒石から右方向に探索し、その先の空きマスを合法手として検出します。



図2-補足5: ビット演算で合法手を発見する過程（右方向、成功例）

b地点の自石から右に1ビットずつシフトしながらAND(opp)を繰り返すと、相手石が連続することが確認できます。相手石の連続の先でAND(empty)が1になったマスが合法手です。**これは「その空きマスに石を置くと相手石を挟める」ことを意味し、合法手として確定します。**

3.5.2 失敗例: 合法手にならない場合

一方、相手石の先に自石がない場合はどうなるでしょうか。



図2-補足6: 自石に到達しない場合（合法手にならない例）

自石から方向にシフトして相手石の連続を確認しますが、その先にAND(empty)=0（空きマスがない）であれば、石を置ける場所がないため合法手にはなりません。プログラム上では svcmpne_u64 の結果が0（FALSE）となり、legalビットには追加されません。

このように、「シフト（移動）→AND(opp)（相手石確認）→繰り返し→AND(empty)（空きマス検出）」の一連のビット操作は、オセロの「自石→相手石列→空きマス」というパターンをビット演算に直接翻訳したものです。この操作をSVE2で全方向同時に実行することで、分岐なしに高速な合法手判定が実現されます。

3.6 なぜ全自石の一斉シフトですべての合法手が求まるのか

3.5節では1つの起点からの探索を例示しましたが、リスト2のコードでは **svdup_u64(my)** でビットボード全体をシフトしています。これは64マスの全自石を同時に1マス進める操作です。この「全自石一斉シフト」でなぜ盤面上のすべての合法手が見つかるのかを説明します。

3.6.1 1マス探索と全自石一斉シフトの違い

素朴な方法では、64マスそれぞれについて8方向を調べるため最悪512回のループが必要です。一方、ビットボードによる全自石一斉シフトでは、**1回のシフト命令で全64ビット（＝全マス）が同時に移動する**ため、8方向×6ステップ＝48回の演算ですべての合法手が同時に確定します。

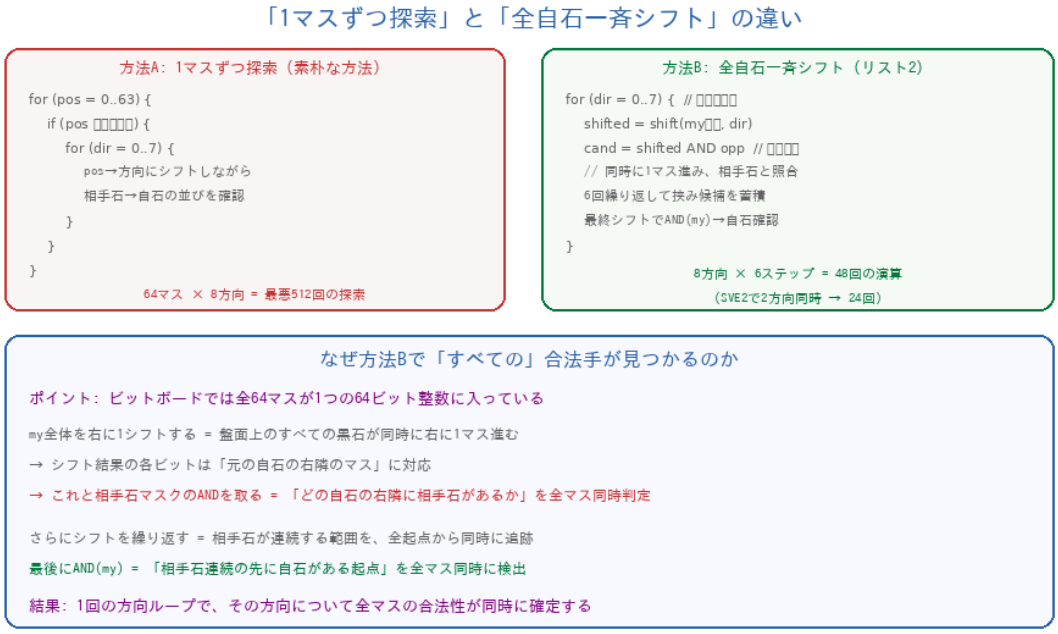


図2-補足7: 1マスずつ探索（方法A）と全自石一斉シフト（方法B）の対比

ビットボードの核心は、**64マスの状態が64ビット整数の各ビットに対応している**ことです。my全体を右に1ビットシフトすると、a列の自石はb列に、b列の自石はc列に……と、盤面上のすべての黒石が同時に右に1マス移動したのと等価になります。

このシフト結果と相手石マスク(opp)のANDを取ると、「どの自石の右隣に相手石があるか」が全マス分、一度に判定されます。さらにシフト＋ANDを繰り返すことで、相手石が何マス連続していても追跡でき、最後に相手石列の先が空きマスかどうかをAND(empty)で確認すれば、すべての合法手が確定します。

3.6.2 具体例: 2つの自石から同時に探索

下図では、a列とe列にある2つの黒石が右方向に同時にシフトされる様子を示します。1回のシフト＋AND操作で、2つの起点からの探索が並列に進みます。



図2-補足8: 2つの自石が同時にシフトされる具体例（右方向）

この例では、a列の黒石は右隣b列が空きマスのためstep1でAND(opp)=0となり脱落します。一方、e列の黒石は右隣f列に白石があるため探索が継続し、f→g（白石連続）→h（空き）へと進みます。**自石が何個あっても同じ回数の演算で完了する点がビットボードの威力です。**

3.6.3 正確なアルゴリズムの全体像

リスト2の概念コードを正確に記述すると、各方向(dir)について3つのステップで構成されます。



図2-補足9: 正確な合法手生成アルゴリズムの全体像

Step 1: 全自石を1マス進め、すぐ隣に相手石があるマスを検出します。**Step 2**: 発見した相手石列をさらに先へ拡張し（最大5回繰り返し）、何マス連続する相手石でも追跡できるようにします。**Step 3**: 相手石連続の先にある空きマスを検出し、それを合法手として確定します。

ここで重要なのは、探索の方向です。コード上では「自石→dir方向に相手石列→その先の空きマス」を検出していますが、これは「空きマスから逆方向に相手石列→自石」の存在と同値です。つまり、**「石を置いたら相手石を挟める空きマス」を全マス同時に見つけている**のです。

8方向すべてのlegalをORで統合すれば、盤面上のすべての合法手のビットマスクが完成します。1つでも方向で挟めるマスは合法手として検出されます。

3.7 EDGE_MASK[8]の導出過程

リスト2のコードには EDGE_MASK[8] という16進定数の配列が登場します。これらの値は「計算で求めた」というより、**8×8盤面の端（辺）の幾何学から直接導出した**ものです。この節では各値の導出過程を説明します。

3.7.0 EDGE_MASKの1と0の意味

EDGE_MASKの各ビットの意味は以下の通りです。

1のビット: そのマス位置のシフト結果をそのまま使う（有効なマス）。シフトしても盤面内に収まっている正しいビットです。

0のビット: そのマス位置のシフト結果を強制的に消す（回り込みで生じた偽ビット）。盤面の端を超えて反対側に回り込んだビットであり、除去しなければ誤った判定になります。

重要な点として、**「1のビットだけをシフトする」のではありません。**処理の順序は以下の通りです。

① まず全64ビットを普通にシフトする ― この時点では回り込みビットが含まれている

② シフト結果に EDGE_MASK を AND する ― マスクの0のビット位置が強制的に0になり、回り込みビットが消える

③ AND後の結果が「正しいシフト結果」として以降の処理に使われる

コード上では **cand = svand_x(pg, shifted, v_mask)** と書かれている部分がこのステップに対応します。shiftedは全ビットシフト後の結果、v_maskがEDGE_MASKです。ANDにより、マスクが0の位置（盤外に回り込んだビット）は消去され、マスクが1の位置（正しいシフト先）だけが残ります。

3.7.1 ビット番号と盤面の対応

ビットボードでは64マスを64ビット整数の各ビットに対応させます。左上a1がbit 0、右に進むとbit番号が増え、h1がbit 7、次行a2がbit 8、右下h8がbit 63です。右に1マス移動=+1ビット、下に1行移動=+8ビット（1行=8マス）となります。



図E-1: ビット番号と盤面マスの対応、各方向のシフト量

ここで問題が生じます。**h列(bit 7)から右に+1シフトすると、bit 8＝次の行のa列に回り込んでしまいます。**この偽のビットを消すためにEDGE_MASKが必要です。

3.7.2 右方向のマスク: 0xFEFEFEFEFEFEFEFE

右方向(+1)にシフトすると、h列の石が次行のa列に回り込む恐れがあります。これを防ぐには、シフト結果に対して「a列のビットをすべて0にする」マスクをANDで適用します。



図E-2: 右方向の端マスク — a列(左端)を除外して回り込みを防止

各行でa列(bit 0)を0にしたバイトは **11111110_2 = 0xFE** です。これが8行分(8バイト)並ぶので、64ビット全体では **0xFEFEFEFEFEFEFEFE** となります。

具体例で確認しましょう。h1(bit 7)に石があるビットボードを右に+1シフトすると、bit 8（＝a2の位置）にビットが立ちます。これは「h1の右隣はa2」という偽の結果です。ここでEDGE_MASK(0xFE...FE)とANDを取ると：

シフト結果: ...0 **1** 0000000 (bit 8 = a2に偽ビット)

EDGE_MASK: ...0 **0** 1111110 (a列=bit 8の位置が0)

AND結果: ...0 **0** 0000000 (偽ビットが消えた！)

このように、AND演算はマスクの0の位置のビットを強制的に0にします。マスクの1の位置のビットはそのまま残ります。これが「EDGE_MASKの1は有効、0は除去」の意味です。

3.7.3 左方向のマスク: 0x7F7F7F7F7F7F7F7F

左方向(-1)では逆にa列の石が前行のh列に回り込みます。h列(各行のbit 7)を0にしたバイト **01111111_2 = 0x7F** を8行分で **0x7F7F7F7F7F7F7F7F**。右方向の0xFEとちょうど左右反転の関係です。

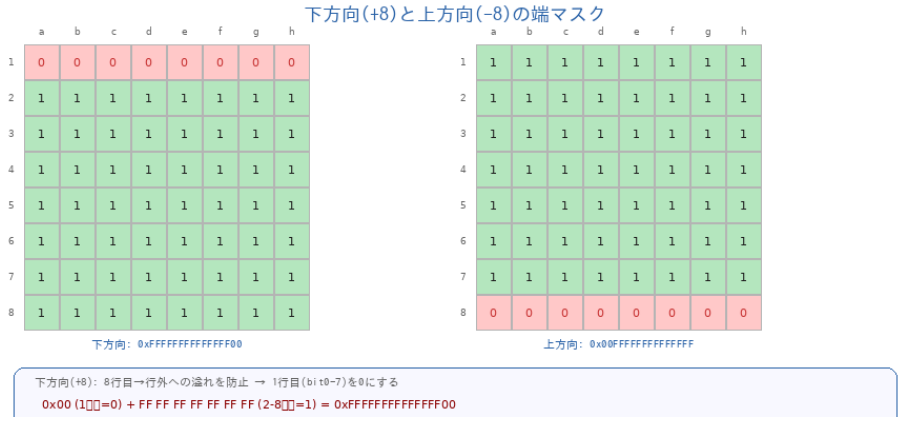


図E-3: 左方向の端マスク — h列(右端)を除外

3.7.4 下方向と上方向のマスク

下方向(+8): 1行目(bit 0-7)を0x00にし残りを0xFF → **0xFFFFFFFFFFFFFF00**

上方向(-8): 8行目(bit 56-63)を0x00にし残りを0xFF → **0x00FFFFFFFFFFFFFF**



図E-4: 下方向と上方向の端マスク — 行単位での除外

3.7.5 斜め方向のマスク: 列除外 AND 行除外の組み合わせ

斜め方向は横成分と縦成分の合成です。列マスクと行マスクの両方を適用するだけで導出できます。



図E-5: 斜め4方向の端マスク — 列除外と行除外の組み合わせ

右下(+9) = 右のa列除外(0xFE)×7行 + 下の1行目除外(0x00) → **0xFEFEFEFEFEFEFE00**

左上(-9) = 上の8行目除外(0x00) + 左のh列除外(0x7F)×7行 → **0x007F7F7F7F7F7F7F**

右上(+7) = h列除外(0x7F)×7行 + 1行目除外(0x00) → **0x7F7F7F7F7F7F7F00**

左下(-7) = 8行目除外(0x00) + a列除外(0xFE)×7行 → **0x00FEFEFEFEFEFEFEULL**

3.7.6 全8方向のまとめ

EDGE_MASK[8] 全8方向のまとめ

idx	方向	除外する辺	マスク値	導出
0	右(+1)	a列(左端)	0xFEFEFEFEFEFEFEFE	□□ 0xFE=11111110
1	左(−1)	h列(右端)	0x7F7F7F7F7F7F7F7F	□□ 0x7F=01111111
2	下(+8)	1行目(上端)	0xFFFFFFFFFFFFFF00	bit0-7=0,□□=1
3	上(−8)	8行目(下端)	0x00FFFFFFFFFFFFFF	bit56-63=0,□□=1
4	右下(+9)	a列+1行目	0xFEFEFEFEFEFEFE00	0xFE×7□+0x00
5	左上(−9)	h列+8行目	0x007F7F7F7F7F7F7F	0x00+0x7F×7□
6	右上(+7)	h列+1行目	0x7F7F7F7F7F7F7F00	0x7F×7□+0x00
7	左下(−7)	a列+8行目	0x00FEFEFEFEFEFEFEULL	0x00+0xFE×7□

共通ルール: 「シフト方向の出口側の辺」を0にしたマスクを作る

図E-6: EDGE_MASK[8] 全8方向のまとめ

共通ルール: **「シフト方向の出口側の辺」を0にしたマスクを作る**。横成分があれば列マスク(0xFEまたは0x7F)を全行に適用し、縦成分があれば行マスク(0x00)を上端または下端に配置します。斜めはこの2つの組み合わせです。

改めて整理すると、EDGE_MASKは**シフト前にフィルタするものではなく、シフト後の結果をクリーンアップするもの**です。全ビットを一斉にシフトした後でANDを取ることにより、盤外への回り込みビットだけを正確に除去し、正しい隣接マスのビットだけを残します。

コード例: SVE2による合法手生成

各方向のシフト量と端マスクをベクトルにロードし、SVE2命令で並列にシフト＋AND処理を行います。

```
#include <arm_sve.h>

static const int      SHIFT_AMOUNTS[8] =
    { 1,-1,8,-8,9,-9,7,-7 };
static const uint64_t EDGE_MASK[8] = {
    0xFEFEFEFEFEFEFEFEULL, 0x7F7F7F7F7F7F7F7FULL,
    0xFFFFFFFFFFFFFF00ULL, 0x00FFFFFFFFFFFFFFULL,
    0xFEFEFEFEFEFEFE00ULL, 0x007F7F7F7F7F7F7FULL,
    0x7F7F7F7F7F7F7F00ULL, 0x00FEFEFEFEFEFEFEull,
};

/*
 * generate_legal_moves_sve2(my, opp)
 *   概要 : SVE2命令で8方向を並列処理し、自石が置ける
 *          すべてのマスをビットマスクとして返す
 *   my   : uint64_t — 自石のビットボード
 *            1のビット位置 = 自石があるマス
 *   opp  : uint64_t — 相手石のビットボード
 *            1のビット位置 = 相手石があるマス
 *   戻値 : uint64_t — 合法手のビットマスク
 *            1のビット位置 = 石を置けるマス
 */
uint64_t generate_legal_moves_sve2(
        uint64_t my, uint64_t opp) {

    /*
     * svptrue_b64()
     *   戻値: svbool_t — 全64bitレーンをTRUEにした述語
     */
    svbool_t pg = svptrue_b64();
    uint64_t legal = 0;

    for (int step = 0; step < 6; step++) {
        for (int d = 0; d < 8; d += 2) {
            svuint64_t v_my  = svdup_u64(my);
            svuint64_t v_opp = svdup_u64(opp);
            svuint64_t v_mask =
                svld1_u64(pg, &EDGE_MASK[d]);

            int s0 = SHIFT_AMOUNTS[d];
            svuint64_t shifted =
                (s0>0) ? svlsr_x(pg,v_my,s0)
                       : svlsl_x(pg,v_my,(uint64_t)(-s0));

            svuint64_t cand = svand_x(pg,shifted,v_opp);
            cand = svand_x(pg,cand,v_mask);

            svbool_t found = svcmpne_u64(pg,
                svand_x(pg,
                    svlsr_x(pg,cand,
                        (uint64_t)(s0>0?s0:-s0)),
                    v_my),
                0);
            legal |= svlastb_u64(found, cand);
        }
    }
```

```
    return legal;
}
```

リスト2: SVE2 intrinsicを用いた合法手生成（概念コード）

リスト2の動作解説

以下の図でgenerate_legal_moves_sve2関数の処理フローをステップごとに解説します。

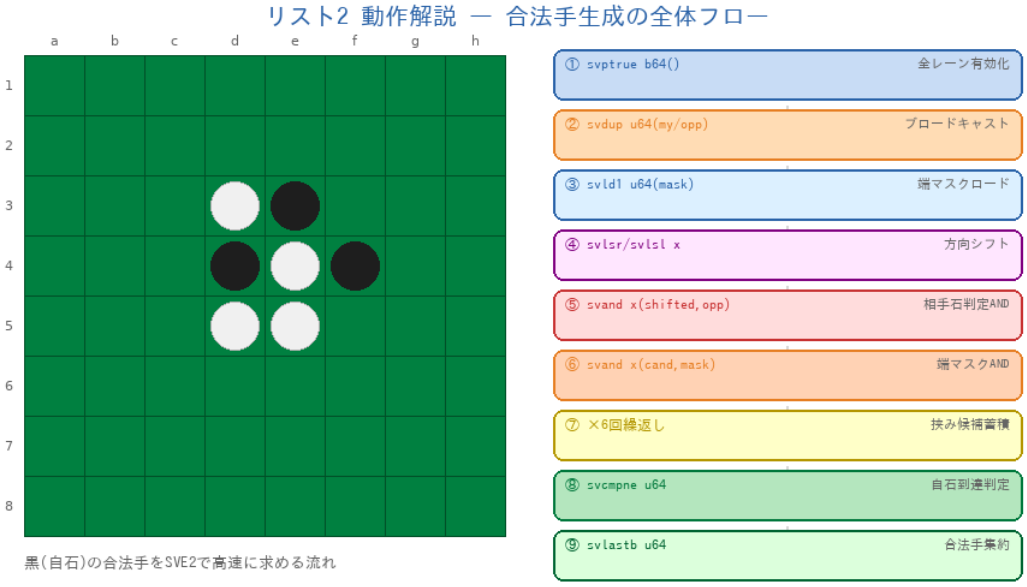


図2a: 合法手生成の全体フロー（9ステップの処理順序）

まず svptrue_b64 で全レーンを有効化した述語を作り、svdup_u64 で自石・相手石のビットボードを全レーンにブロードキャストします。次に svld1_u64 で端マスク配列から2方向分のマスクを同時にロードします。

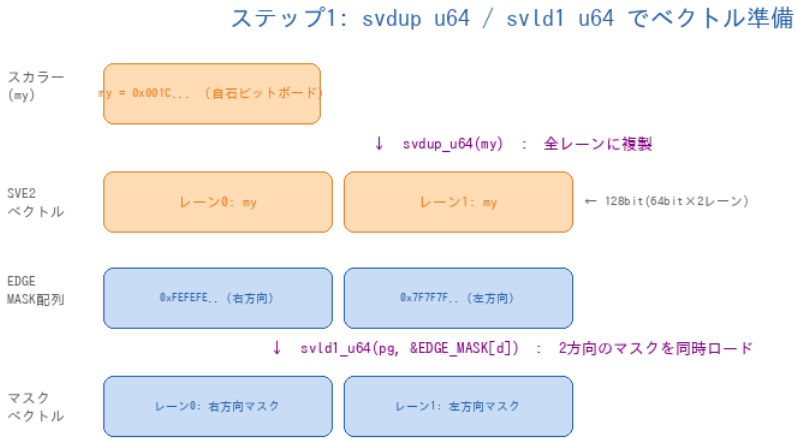


図2b: ステップ1 ― svdup_u64 / svld1_u64 でベクトルにデータを準備

128ビットレジスタは64ビット×2レーン構成のため、1回のSVE2演算で2方向を同時処理できます。4回のペア処理（d=0,2,4,6）で全8方向をカバーします。



図2c: ステップ2 — 1方向のシフト → AND → 挟み判定の連鎖

svlsr_x で自石ビットを方向に沿ってシフトし、svand_x で相手石との交差を判定します。さらに端マスクとの AND で盤外への回り込みを防止します。このシフト＋AND を最大6回繰り返し、相手石の連続範囲を特定した後、svcmpne_u64 で自石に到達したかを判定して合法手を確定します。



図2d: ステップ3 — 2レーン並列処理と合法手集約

4ペア（右+左、下+上、右下+左上、右上+左下）の処理が完了したら、各ペアの結果をsvlastb_u64 でスカラーに集約し、ビットOR で legal 変数に統合します。スカラー処理の48回のループが、SVE2では24回のSIMD演算に削減されます。

◆ SIMD命令 詳細解説

[1] ― 全64bitレーンをTRUEにした述語レジスタを返します。以降の全演算はこの述語が有効なレーン全体を対象とします。

[2] ― スカラー値を全レーンに複製(ブロードキャスト)したベクトルを生成。my/oppの値を2レーン同時に保持します。

[3] ― 述語pgが有効なレーンのみメモリからロード。端マスク配列の2要素を1命令でベクトルに取り込みます。

[4] ― 論理右シフト(Logical Shift Right)。pgが有効な全レーンを同一シフト量で一斉にシフト。_xは非アクティブレーンが未定義でよいことを示し最速実行されます。

[5] ― 論理左シフト(Logical Shift Left)。負方向(右→左)の方向探索に使用します。

[6] ― ビットAND。1回目は相手石マスクとのANDで「シフト先が相手石か」を判定。2回目は端マスクとのANDで盤外への回り込みを防止します。両者とも全レーン同時処理です。

[7] ― 各レーンをimmと比較し、≠なら述語TRUEを生成。「自石に到達したか」を条件分岐なしに全レーン同時判定します。

[8] ― 述語pgが最後にTRUEとなったレーンの値をスカラーで返します。合法手ビットの集約に使用します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svptrue_b64	svbool_t	(引数なし)	全64bitレーンをTRUE(1)にした述語レジスタを返す。演算対象を全レーンとする最もシンプルな述語。
svdup_u64	svuint64_t	uint64_t imm / scalar	スカラー値をSVE2ベクトルの全レーンに複製(ブロキャスト)したベクトルを返す。
svld1_u64	svuint64_t	svbool_t pg, const uint64_t *ptr	pgがTRUEのレーンのみptrからロード。FALSEレーは0で埋める。境界外アクセスを防ぐ述語付きロー
svlsr_x	svuint64_t	svbool_t pg, svuint64_t op, uint64_t shift	pgが有効なレーンをshiftビット分論理右シフト。_xinactiveレーンが未定義=最高速。正方向(右)探索に応。
svlsl_x	svuint64_t	svbool_t pg, svuint64 t op, uint64_t shift	pgが有効なレーンをshiftビット分論理左シフト。負(左)探索に対応。svlsr_xと対をなす命令。
svand_x	svuint64_t	svbool t pg, svuint64 t a, svuint64_t b	pgが有効なレーンのa&bをビットAND。相手石判定マスク適用の両方に使用する汎用AND命令。

svcmpne_u64	svbool_t	svbool_t pg, svuint64_t a, svuint64_t b	pgが有効な各レーンでa≠bを比較し結果を述語とします。0との比較で「ビットが立っているか」を判定。
svlastb_u64	uint64_t	svbool_t pg, svuint64_t v	pgが最後にTRUEのレーンの値をスカラーで返す。手ビットをスカラーに集約するために使用。

4. 石の反転処理（フリップ）

石を置いた後、挟まれた相手の石を裏返す「フリップ処理」は、オセロ思考ルーチンで最も頻繁に呼ばれる関数です。探索中に何百万回も実行されるため、1命令の差が全体の性能に大きく影響します。

4.1 シフト＋ANDの連鎖

フリップ処理の基本アルゴリズムは、自石の位置から特定方向にビットシフトし、相手石マスクとANDを取ることで「相手石が連続する範囲」を特定するものです。これを最大6回繰り返します。

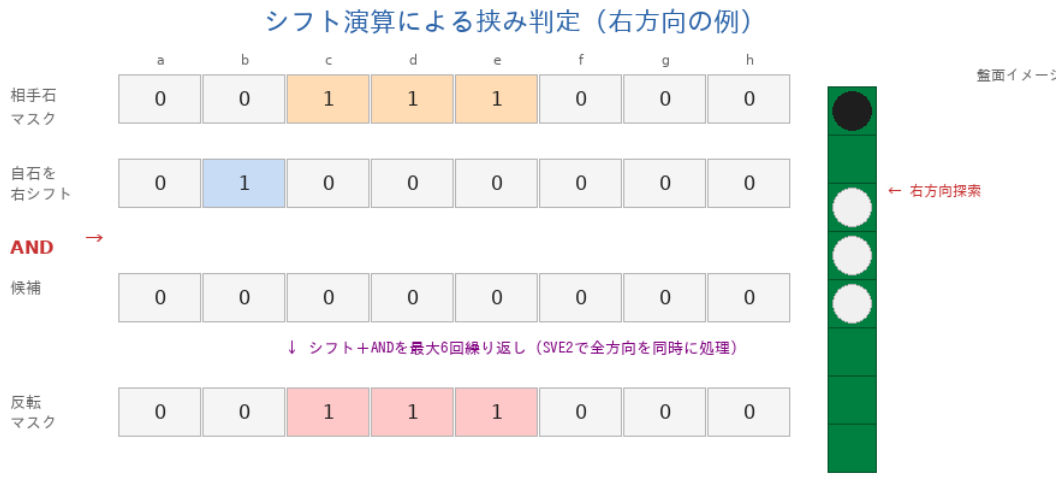


図3: シフト＋AND演算による挟み判定の流れ（右方向の例）

4.2 SVE2命令の活用ポイント

SVE2では、シフト量を各レーンで個別に指定できる可変シフト命令が利用可能です。これにより、8方向で異なるシフト量を1命令で同時適用できます。

コード例: フリップ処理とNEON盤面更新

全方向のフリップマスクを求めた後、NEONのveorq_u64命令で黒白ビットボードを同時にXOR反転します。

```
#include <arm_sve.h>
#include <arm_neon.h>

/*
 * flip_discs_sve2(my, opp, pos)
 *   概要 : 着手位置posに石を置いたとき、挟まれて
 *          裏返る相手石の位置をビットマスクで返す
 *   my   : uint64_t — 自石のビットボード
```

```
 *   opp  : uint64_t — 相手石のビットボード
 *   pos  : int      — 着手するマス番号 (0=左上, 63=右下)
 *   戻値 : uint64_t — 反転する相手石のビットマスク
 *            1のビット位置 = 裏返すべき相手石の位置
 */
uint64_t flip_discs_sve2(uint64_t my, uint64_t opp,
                         int pos) {
    uint64_t flip_mask = 0;
    uint64_t placed    = 1ULL << pos;

    for (int dir = 0; dir < 8; dir++) {
        int      shift = SHIFT_AMOUNTS[dir];
        uint64_t mask  = EDGE_MASK[dir];
        uint64_t flip  = 0, cursor = placed;

        for (int i = 0; i < 6; i++) {
            /*
             * ビットシフト + 端マスクAND
             *   cursor : uint64_t — 現在のカーソル位置ビット
             *   shift  : int — 正値=左シフト / 負値=右シフト
             *   mask   : uint64_t — 端マスク(盤外回り込み防止)
             */
            cursor = (shift > 0)
                ? (cursor << shift) & mask
                : (cursor >> -shift) & mask;
            if (!(cursor & opp)) break;
            flip |= cursor;
        }
        uint64_t beyond = (shift > 0)
            ? (cursor << shift) & mask
            : (cursor >> -shift) & mask;
        if (beyond & my) flip_mask |= flip;
    }
    return flip_mask;
}

/*
 * apply_flip_neon(board, flip, placed)
 *   概要 : NEONのXOR命令で黒白ビットボードを
 *          1命令で同時更新し次局面を生成する
 *   board  : uint64x2_t — 現在の盤面SIMDレジスタ
 *              レーン0 = 黒ビットボード
 *              レーン1 = 白ビットボード
 *   flip   : uint64_t   — 反転する相手石のビットマスク
 *   placed : uint64_t   — 着手した自石のビット(1ULL<<pos)
 *   戻値   : uint64x2_t — 更新後の盤面SIMDレジスタ
 */
inline uint64x2_t apply_flip_neon(
        uint64x2_t board,
        uint64_t flip, uint64_t placed) {
    uint64_t data[2] = { flip | placed, flip };
    /*
     * vld1q_u64(ptr) / veorq_u64(a, b)
     *   黒レーン: flip|placed (自石追加+裏返し)
     *   白レーン: flip        (裏返された相手石を消す)
     *   ※ XOR 1命令で黒白2ボードを同時更新
     */
    return veorq_u64(board, vld1q_u64(data));
}
```

リスト3: フリップ処理とNEON XORによる盤面更新

◆ SIMD命令 詳細解説

[1][2] cursor/beyondのシフト+AND ― C++の << / >> とビット論理ANDを組み合わせた処理。方向(正=左シフト/負=右シフト)に応じてカーソルを進め、端マスクで盤外を除去します。最大6回繰り返して挟み候補を蓄積します。

[3] ― 黒レーン用(flip|placed)と白レーン用(flip)の2値を128ビットレジスタに一括ロード。レーン別の反転マスクを1命令で準備します。

[4] ― NEON EOR命令(Quadword/64bit lanes)。黒ビットボード・白ビットボードを1命令で同時にXOR反転。 XORにより反転マスクが1のビット位置の石が黒白入れ替わります。「石を置く＋挟んだ石を裏返す」が1命令で完結します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vld1q_u64	uint64x2_t	const uint64_t *ptr	メモリから64bit×2要素(128bit)を1命令でロード。黒レーン用(flip|placed)と白レーン用(flip)を同時にベクト化する。
veorq_u64	uint64x2_t	uint64x2_t a, uint64x2_t b	a、bの各対応レーンをビットXOR。黒ビットボード白ビットボードを1命令で同時反転。石を置く+挟ん石を裏返す操作が1命令で完結する。
vst1q_u64	void	uint64_t *ptr, uint64x2_t val	128bitをメモリへ一括ストア。EOR後の次局面ビードを構造体へ書き出す。

5. 述語レジスタによる合法手フィルタリング

SVE2の最大の特徴の1つが述語（プレディケート）レジスタです。これはベクトルの各レーンに対して「処理する／しない」を示すビットマスクで、条件分岐なしにデータの選択的処理を実現します。

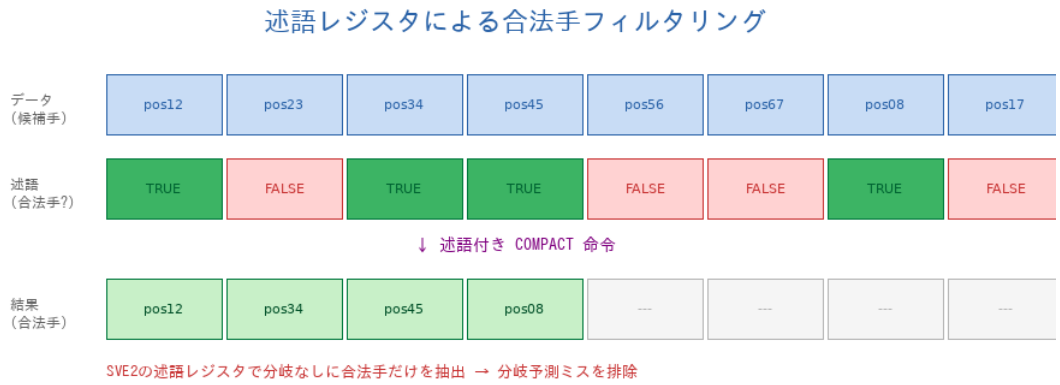


図4: 述語レジスタによる合法手の選択的抽出

従来のスカラー処理では**if文による分岐**で合法手を判定していました。分岐予測ミスによるパイプラインストールを、SVE2の述語付き命令で**完全に排除**できます。

コード例: SVE2 COMPACT命令による合法手抽出

WHILELT命令で残り要素数に応じた述語を自動生成し、COMPACT命令で合法手だけを詰め込みます。

```
#include <arm_sve.h>

/*
 * filter_legal_moves_sve2(candidates, legal_out,
 *                         count, legal_mask)
 *   概要 : SVE2の述語レジスタとCOMPACT命令を使い、
 *          候補手の中から合法手だけをif分岐なしで抽出する
 *   candidates : const uint8_t* — 候補手座標の配列
 *                  各要素は盤面マス番号(0〜63)
 *   legal_out  : uint8_t*       — 合法手座標の出力バッファ
 *                  呼び出し元が十分なサイズを確保すること
 *   count      : int            — candidates の要素数
 *   legal_mask : uint64_t       — 合法手ビットマスク
 *                  1のビット位置 = 合法手のマス番号
 *   戻値       : int            — 抽出した合法手の個数
 */
int filter_legal_moves_sve2(
        const uint8_t* candidates,
        uint8_t*       legal_out,
        int            count,
        uint64_t       legal_mask) {
    int out_idx = 0, i = 0;
```

```
    while (i < count) {
        /*
         * svwhilelt_b8(base, limit)
         *   base : int64_t — 現在インデックス(i)
         *   limit: int64_t — 上限インデックス(count)
         *   戻値 : svbool_t — 残り要素数に応じた述語を生成
         */
        svbool_t pg = svwhilelt_b8(i, count);

        svuint8_t pos = svld1_u8(pg, &candidates[i]);

        svuint8_t bits = svlsr_x(pg,
            svreinterpret_u8(svdup_u64(legal_mask)), pos);

        svuint8_t is_legal =
            svand_x(pg, bits, svdup_u8(1));

        svbool_t pg_legal = svcmpne_u8(pg, is_legal, 0);

        /*
         * svcompact_u8(pg, v)     ← SVE2 COMPACT命令
         *   pg  : svbool_t  — 抽出対象(合法手=TRUE)の述語
         *   v   : svuint8_t — 候補手座標ベクトル
         *   戻値: svuint8_t — TRUEレーンの要素を先頭に詰めた結果
         */
        svuint8_t compacted = svcompact_u8(pg_legal, pos);

        int n = svcntp_b8(pg_legal, pg_legal);
        svst1_u8(svwhilelt_b8(0, n),
                 &legal_out[out_idx], compacted);
        out_idx += n;
        i += svcntb();
    }
    return out_idx;
}
```

リスト4: SVE2述語レジスタとCOMPACT命令による合法手フィルタリング

◆ SIMD命令 詳細解説

[1] ― インデックスiからcountまでの残り要素数に応じた述語を自動生成。ループ末尾の端数処理が不要になります。

[2] ― 述語が有効なレーンのみバイト単位でロード。境界外アクセスが発生しません。

[3] ― 可変シフト。各レーンで異なるシフト量(各候補座標の値)を使い、legal_maskの対応ビットを最下位へ引き出します。全候補の合法判定が1命令で完了します。

[4] ― 最下位ビットだけをマスクして取り出します。0なら違法・1なら合法の判定値になります。

[5] ― 0でないレーン(合法手)をTRUEとする述語を生成。この述語が後続COMPACT命令の入力になります。

[6] ― **SVE2 COMPACT命令。**pgがTRUEのレーンの要素だけをベクトルの先頭に詰めます。if分岐なしにスカラーのフィルタループを完全代替する強力な命令です。

[7] ― 述語内のTRUEビット数をカウント。今回処理した合法手の個数を求め、出力インデックスを正確に進めます。

[8] ― 述語付きバイトストア。有効レーンのみ出力バッファへ書き込みます。

[9] ― 現在のSVE2実装のベクトル長(バイト数)を実行時に返します。DGX Sparkでは16(128bit÷8)を返します。このVLA設計で将来の広幅実装でも同一コードが動きます。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svwhilelt_b8	svbool_t	int64_t base, int64_t limit	base < limitの範囲内で8bitレーンをTRUEにした述語を返す。ループの残り要素数に応じた述語を自動生成。ディング不要にする。
svld1_u8	svuint8_t	svbool_t pg, const uint8_t *ptr	pgがTRUEのレーンのみ1バイトずつロード。境界外クセスなしに端数要素を安全に処理できる。
svlsr_x	svuint8_t	svbool t pg, svuint8 t op, svuint8_t shift	各レーン個別のシフト量(pos値)で右シフト。可変シトにより全候補座標の合法判定を1命令で実行できる
svdup_u8	svuint8_t	uint8_t imm	8bitスカラー値を全レーンにブロードキャスト。マ値(1)の配布や比較基準値の準備に使用。
svand_x	svuint8_t	svbool_t pg, svuint8_t a, svuint8_t b	ビットAND。最下位ビットだけを取り出し合法(1)/(0)を判定する。
svcmpne_u8	svbool_t	svbool_t pg, svuint8_t a, uint8_t imm	各レーンでa≠immを比較し述語を返す。0との比較で法手レーンをTRUEにした述語を生成する。
svcompact_u8	svuint8_t	svbool_t pg, svuint8_t v	pgがTRUEのレーンの要素だけを先頭に詰める。if分不要で合法手の前詰め処理を1命令で実現するSVE2有命令。
svcntp_b8	uint64_t	svbool_t pn, svbool_t pm	pmのうちpnがTRUEの位置のTRUEビット数をカウト。今回処理した合法手の個数を求め出力インデックを正確に進める。
svst1_u8	void	svbool_t pg, uint8_t *ptr, svuint8_t v	pgがTRUEのレーンのみ1バイトずつストア。合法手けを出力バッファへ書き込む境界安全なストア命令
svcntb	uint64_t	(引数なし)	現在のSVE2実装のベクトル長(バイト数)を実行時にす。128bit実装では16。VLA設計でハードウェアにしないループ制御が可能。

6. 評価関数のベクトル化

探索中の各局面に対して「どちらが有利か」を数値で判定する評価関数は、思考ルーチンの知性の核です。典型的なオセロ評価関数では、盤面の各位置に重みを持つ評価テーブルを用い、特徴量との積和（内積）を計算します。

6.1 MLA命令による積和演算

SVE2のMLA（Multiply-Accumulate）命令は、乗算と加算を1命令で実行します。64マスの評価をベクトル長ぶんずつ処理すれば、わずか数回のMLA命令で完了します。

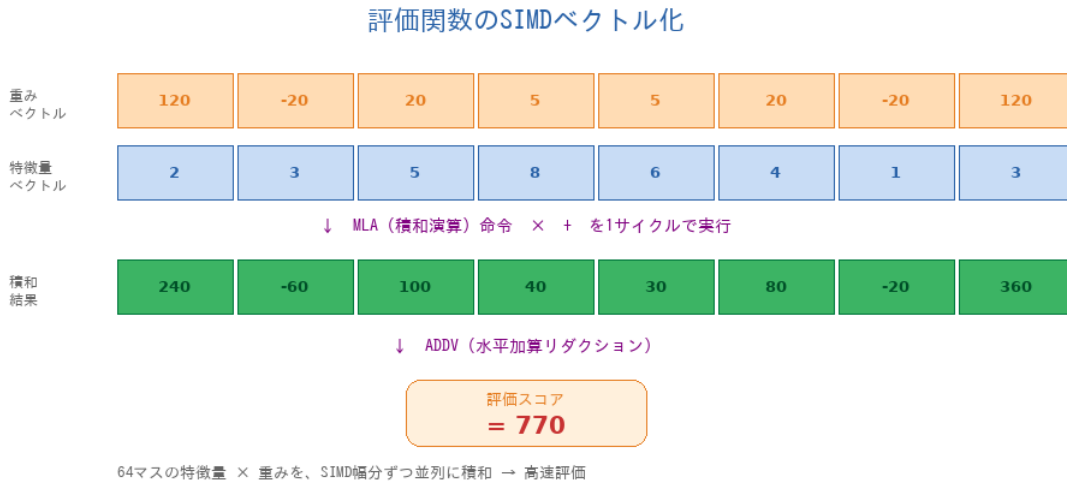


図5: MLA命令とADDVリダクションによる評価関数の高速計算

6.2 ADDV水平リダクション

MLA命令で各レーンに部分和が求まった後、SVE2のADDV命令でベクトル内の全要素を1命令で合算して最終スコアを得ます。

コード例: SVE2 MLA + ADDV による評価関数

64マス分の重みと特徴量をベクトル長ずつロードし、svmla命令で積和した結果をsvaddvでリダクションします。

```
#include <arm_sve.h>

static const int16_t EVAL_WEIGHTS[64] = {
    120,-20, 20,  5,  5, 20,-20,120,
    -20,-40, -5, -5, -5, -5,-40,-20,
     20, -5, 15,  3,  3, 15, -5, 20,
      5, -5,  3,  3,  3,  3, -5,  5,
      5, -5,  3,  3,  3,  3, -5,  5,
     20, -5, 15,  3,  3, 15, -5, 20,
    -20,-40, -5, -5, -5, -5,-40,-20,
```

```
    120,-20, 20,  5,  5, 20,-20,120,
};

/*
 * evaluate_sve2(my, opp)
 *   概要 : SVE2のMLA命令(積和)とADDV命令(水平リダクション)で
 *          盤面の評価スコアを高速計算して返す
 *   my   : uint64_t — 自石のビットボード
 *            評価テーブルで重みに +1 を乗じるマス
 *   opp  : uint64_t — 相手石のビットボード
 *            評価テーブルで重みに -1 を乗じるマス
 *   戻値 : int32_t  — 盤面評価スコア
 *            正値 = 自石が有利 / 負値 = 相手が有利
 */
int32_t evaluate_sve2(uint64_t my, uint64_t opp) {
    svint16_t acc = svdup_s16(0);   // アキュムレータを0初期化

    int16_t features[64];
    for (int i = 0; i < 64; i++) {
        uint64_t bit = 1ULL << i;
        features[i] = (my&bit)?1 : (opp&bit)?-1 : 0;
    }

    for (int i = 0; i < 64; i += svcnth()) {
        svbool_t  pg = svwhilelt_b16(i, 64);
        svint16_t w  = svld1_s16(pg, &EVAL_WEIGHTS[i]);
        svint16_t f  = svld1_s16(pg, &features[i]);
        /*
         * svmla_s16_x(pg, acc, w, f)  ← MLA命令
         *   pg  : svbool_t  — 処理対象レーンの述語
         *   acc : svint16_t — アキュムレータ(累積先)
         *   w   : svint16_t — 位置重み(EVAL_WEIGHTS)
         *   f   : svint16_t — 特徴量(+1 / 0 / -1)
         *   戻値: svint16_t — acc += w×f の結果
         */
        acc = svmla_s16_x(pg, acc, w, f);
    }

    /*
     * svaddv_s16(pg, v)  ← ADDV命令 (水平リダクション)
     *   pg  : svbool_t  — 加算対象の述語(全レーン)
     *   v   : svint16_t — 各レーンの部分積和(acc)
     *   戻値: int16_t   — 全レーンの総和スカラー
     */
    int16_t score = svaddv_s16(svptrue_b16(), acc);
    return (int32_t)score;
}
```

リスト5: SVE2 MLA積和 + ADDV水平加算による評価関数

◆ SIMD命令 詳細解説

[1] — 全レーンを符号付き16bit値0で初期化したベクトルを生成。MLA命令の累積アキュムレータをゼロクリアします。

[2] — 現在実装のSVE2ベクトル長をハーフワード(16bit)数で返します。128bit実装では8を返し、ループステップ幅として使います。

[3] — 残りマス数に応じた述語を生成。64が16の倍数でない場合も端数を自動処理しパディング不要です。

[4] ― 述語が有効なレーンのみ符号付き16bitをロード。重みテーブルと特徴量を各1命令でベクトルへ取り込みます。

[5] ― **SVE2 MLA命令(Multiply-Accumulate)。** acc += w × f を乗算+加算の1命令で実行。スカラーなら1マス2命令が必要なところをベクトル長分(128bit時8要素)まとめて処理します。

[6] ― **SVE2 ADDV命令(Add Across Vector)。** ベクトル全レーンを1命令で合算しスカラーを返す水平リダクション。スカラーでは要素数-1回の加算が必要な処理が1命令で完了します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
svdup_s16	svint16_t	int16_t imm	符号付き16bit値を全レーンに複製。MLA命令のアキュムレータ初期化(0クリア)に使用する。
svcnth	uint64_t	(引数なし)	現在実装のSVE2ベクトル長をハーフワード(16bit)数で返す。128bit実装では8。ループステップ幅の決定に使用。
svwhilelt_b16	svbool_t	int64 t base, int64_t limit	base < limitの範囲で16bitレーンをTRUEにした述語を返す。64マスの端数処理を自動化し境界チェックを排除する。
svld1_s16	svint16_t	svbool t pg, const int16_t *ptr	pgがTRUEのレーンのみ符号付き16bitをロード。重みテーブル・特徴量を各1命令でベクトルに取り込む。
svmla_s16_x	svint16_t	svbool_t pg, svint16_t acc, svint16 t w, svint16_t f	acc += w × f を乗算+加算の1命令で実行するMLA(Multiply-Accumulate)命令。128bit時に8要素をサイクルで処理。_xはinactiveレーン未定義=最速。
svaddv_s16	int16_t	svbool_t pg, svint16_t v	pgが有効な全レーンを合算しスカラーを返す水平リダクション(Add Across Vector)。要素数-1回の加算が1命令で完了する。
svptrue_b16	svbool_t	(引数なし)	全16bitレーンをTRUEにした述語を返す。svaddv_s16に全レーンを対象として渡す際に使用する。

7. ポピュレーションカウント（石数計算）

ビットボードで「1が立っているビットの数」をカウントするポピュレーションカウント（popcount）は、石の数や着手可能数の計算に不可欠です。



図6: CNT命令によるビットカウントとその応用

SVE2/NEONのCNT命令は、ビットボード内の1のビット数を高速に数えます。終局時の勝敗判定、mobility評価、パターンマッチングなど、あらゆる場面で使用されます。

コード例: NEON/SVE2 CNT命令による石数カウント

NEON版ではvcnt_u8でバイト単位のビットカウント後にvaddlv_u8で総和します。SVE2版ではsvcnt + svaddvで同様の処理を行います。

```
#include <arm_neon.h>
#include <arm_sve.h>

/*
 * popcount_neon(bits)
 *   概要 : NEON CNT命令で64bitビットボードの
 *          1ビット数(立っているビットの個数)を高速カウントする
 *   bits : uint64_t — カウント対象のビットボード
 *   戻値 : int      — 1ビットの個数 (0〜64)
 */
inline int popcount_neon(uint64_t bits) {
    uint8x8_t v = vreinterpret_u8_u64(
                      vcreate_u64(bits));
    /*
     * vcnt_u8(a) ← CNT命令
     *   a    : uint8x8_t — 8バイトの入力ベクトル
     *   戻値 : uint8x8_t — 各バイトのビット数を並列カウント
     */
    uint8x8_t cnt = vcnt_u8(v);
    /*
     * vaddlv_u8(a) ← ADDLV命令
     *   a    : uint8x8_t — CNT結果ベクトル
     *   戻値 : uint16_t  — 全8バイトのビット数の総和
     */
    return (int)vaddlv_u8(cnt);
}
```

```
/*
 * popcount_sve2(bits)
 *   概要 : SVE2 CNT+ADDV命令でpopcount_neonと同等の処理を
 *          可変長ベクトル対応で実装したSVE2版
 *   bits : uint64_t — カウント対象のビットボード
 *   戻値 : int      — 1ビットの個数 (0〜64)
 */
inline int popcount_sve2(uint64_t bits) {
    svbool_t  pg = svptrue_b8();
    svuint8_t v  = svld1_u8(pg,
        reinterpret_cast<const uint8_t*>(&bits));
    svuint8_t cnt = svcnt_u8_x(pg, v);
    return (int)svaddv_u8(pg, cnt);
}

/*
 * mobility(legal_mask)
 *   概要 : 合法手ビットマスクのbitcountで
 *          現局面の着手可能数(mobility)を返す
 *   legal_mask : uint64_t — 合法手ビットマスク
 *   戻値       : int      — 着手可能手数 (0〜64)
 */
inline int mobility(uint64_t legal_mask) {
    return popcount_neon(legal_mask);
}

struct GameResult { int black, white, winner; };

/*
 * judge_game(b)
 *   概要 : 終局後の盤面から黒白の石数を数え
 *          勝敗を判定して返す
 *   b      : const Board& — 終局盤面
 *   戻値   : GameResult
 *     .black  : int — 黒石の個数
 *     .white  : int — 白石の個数
 *     .winner : int — 1=黒勝ち / -1=白勝ち / 0=引き分け
 */
GameResult judge_game(const Board& b) {
    GameResult r;
    r.black  = popcount_neon(b.black);
    r.white  = popcount_neon(b.white);
    r.winner = (r.black > r.white) ?  1 :
               (r.white > r.black) ? -1 : 0;
    return r;
}
```

リスト6: NEON/SVE2によるpopcount実装と終局判定

◆ SIMD命令 詳細解説

[1] — NEON CNT命令。8バイトのベクトル各バイトについて立っているビット数を並列カウント。64bitを8バイトに分割して処理することで8バイト同時カウントを実現します。

[2] — NEON ADDLV命令(Add Long across Vector)。8つの8bit値を16bitに拡張しながら総和。CNT結果を合算してpopcount最終値を得ます。

[3] — SVE2版CNT命令。各バイトのビット数を述語付きで並列カウント。可変長に対応し将来の広幅実装でも同一コードで動作します。

[4] ― SVE2 ADDV。全バイトのカウントを1命令で総和してスカラー返却。NEON版vaddlv_u8に相当し述語で有効範囲を制御できます。

[5][6] popcount_neon応用 ― mobility計算には合法手マスクのbitcount、終局判定には黒・白ビットボードのbitcountをそれぞれ1関数呼び出しで即座に取得します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
vcreate_u64	uint64x1_t	uint64_t a	スカラーuint64_tを64bit幅のNEONベクトル型に変換 vreinterpret系との組み合わせで型変換チェーンの起点となる。
vreinterpret_u8_u64	uint8x8_t	uint64x1_t a	64bit値を8×8bitとして再解釈(ビット変換なし)。vcnt_u8が8bit粒度を要求するための型キャスト。
vcnt_u8	uint8x8_t	uint8x8_t a	8バイトの各バイトについてビット数を並列カウントするCNT命令。64bitを8バイトに分割することで8バイト同時カウントを実現。
vaddlv_u8	uint16_t	uint8x8_t a	8つの8bit値を16bitに拡張しながら横加算するADDLV命令。vcnt_u8の結果を合算してpopcount最終値を得る。
svcnt_u8_x	svuint8_t	svbool_t pg, svuint8_t v	SVE2版CNT命令。各バイトのビット数を述語付きで並列カウント。可変長ベクトルに対応し将来の広幅実装も同一コードで動作する。
svaddv_u8	uint8_t	svbool_t pg, svuint8_t v	SVE2 ADDV。全バイトのカウントを1命令で総和しスカラーを返す水平リダクション。述語で有効範囲を制御できる。
svld1_u8	svuint8_t	svbool_t pg, const uint8_t *ptr	述語が有効なレーンのみバイト単位でロード。64bitビットボードを8バイト単位でSVE2ベクトルとして取り込む。

8. SIMD活用による性能向上

各処理でSIMDを活用した場合の高速化効果のイメージを以下に示します。

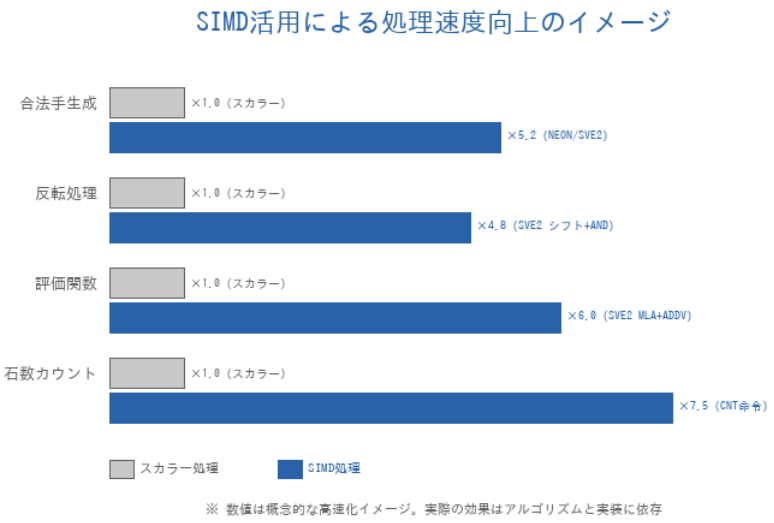


図7: 各処理におけるSIMD高速化のイメージ（概念図）

8.1 主要処理と対応SIMD命令

処理	使用するSIMD命令	効果
合法手生成	SHL/SHR, AND, ORR (×8レーン)	8方向同時探索
反転処理	可変シフト, AND, BIC	全方向同時フリップ
評価関数	MLA, ADDV	積和＋リダクション一括
石数カウント	CNT	popcount 1命令
合法手選択	COMPACT + 述語	分岐レス抽出
着手可能数	CNT + ADDV	mobility高速計算

8.2 DGX Sparkの実行スループット

DGX SparkのCortex-X925（Pコア）は128ビットのNEON/SVE2命令を1サイクルに最大6個リタイアできます。Cortex-A725（Eコア）でも2個リタイア可能です。

9. DGX Spark上のオセロAI全体像

DGX Sparkの最大の特徴は、CPU（ARM SIMD）とGPU（Blackwell）が128GBの統一メモリを共有するUMAアーキテクチャです。CPU側のSIMD処理による高速な盤面操作・探索と、GPU側のTensor Coreによるニューラルネットワーク評価を、メモリコピーなしで組み合わせることが可能です。

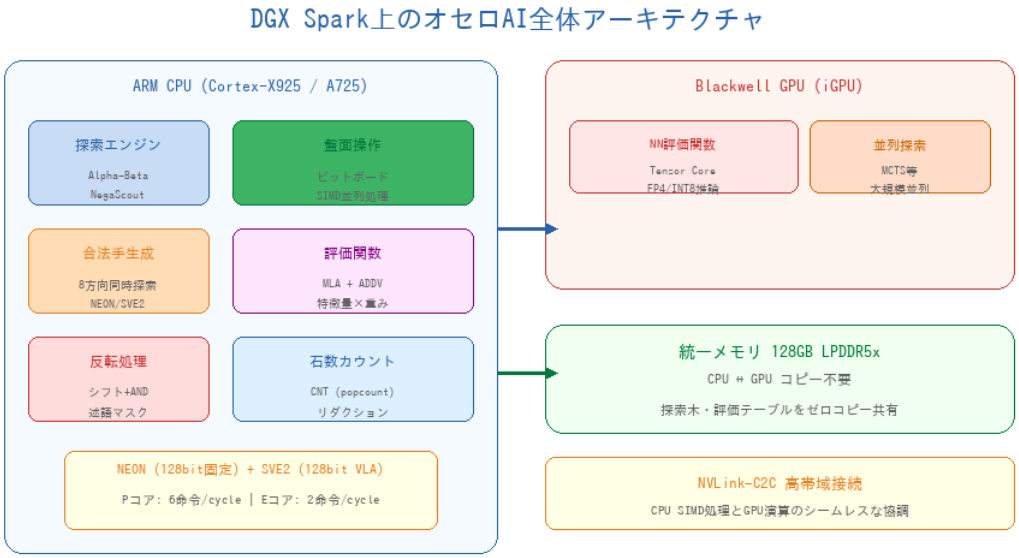


図8: DGX Spark上のオセロAI全体アーキテクチャ

コード例: SIMD高速探索エンジン

ここまで解説した各SIMD関数を組み合わせたAlpha-Beta探索の全体像です。

```
#include <arm_neon.h>

/*
 * alpha_beta(board, depth, alpha, beta, my_turn)
 *   概要 : SIMD命令で高速化したAlpha-Beta木探索。
 *          合法手生成・フリップ・評価をすべてSIMD関数で処理する
 *   board   : Board — 探索開始局面の盤面
 *   depth   : int   — 残り探索深さ(0になったら評価関数を呼ぶ)
 *   alpha   : int   — Alphaカット値(現在の自分側の下限スコア)
 *   beta    : int   — Betaカット値(現在の相手側の上限スコア)
 *   my_turn : bool  — true=自石手番 / false=相手石手番
 *   戻値    : int   — この局面の最善スコア
 *               正値 = 自石有利 / 負値 = 相手有利
 */
int alpha_beta(Board board, int depth,
               int alpha, int beta, bool my_turn) {

    // 深さ0: SVE2評価関数で局面スコアを計算して返す
    if (depth == 0)
        return evaluate_sve2(board.black, board.white);

    uint64_t my  = my_turn ? board.black : board.white;
```

```
    uint64_t opp = my_turn ? board.white : board.black;

    // SVE2並列合法手生成
    uint64_t legal =
        generate_legal_moves_sve2(my, opp);

    if (legal == 0)    // 合法手なし → パス
        return -alpha_beta(board, depth-1,
                           -beta, -alpha, !my_turn);

    int best = -999999;
    while (legal) {
        /*
         * __builtin_ctzll(x)
         *   x    : unsigned long long — 合法手ビットマスク
         *   戻値 : int — 最下位1ビットの位置(着手マス番号)
         */
        int pos = __builtin_ctzll(legal);
        legal &= legal - 1;

        // flip_discs_sve2(): 裏返す石のビットマスクを取得
        uint64_t flip   = flip_discs_sve2(my, opp, pos);
        uint64_t placed = 1ULL << pos;

        uint64_t fdata[2] = {
            my_turn ? flip | placed : flip,
            my_turn ? flip          : flip | placed };

        /*
         * vld1q_u64 / veorq_u64 / vst1q_u64
         *   現局面(board)を128bitにロード → XORで次局面生成
         *   → next構造体に1命令でストア
         *   3命令だけで次局面の盤面生成が完結する
         */
        uint64x2_t bw = vld1q_u64((uint64_t*)&board);
        bw = veorq_u64(bw, vld1q_u64(fdata));
        Board next;
        vst1q_u64((uint64_t*)&next, bw);

        int val = -alpha_beta(next, depth-1,
                              -beta, -alpha, !my_turn);
        if (val > best) best = val;
        if (val > alpha) alpha = val;
        if (alpha >= beta) break;    // β枝刈り
    }
    return best;
}
```

リスト7: SIMD活用Alpha-Beta探索エンジン

◆ SIMD命令 詳細解説

[1] ― 葉ノードで第6章のSVE2 MLA+ADDV評価関数を呼び出します。探索末端で大量に呼ばれるため、この関数のSIMD化が全体性能に最も貢献します。

[2] ― 第3章の8方向SIMD並列合法手生成。探索中に何百万回も呼ばれる最頻出処理です。

[3] ― GCC/Clang組み込み関数「Count Trailing Zeros(64bit)」。合法手マスクの最下位ビット位置(候補手座標)を1命令で取得します。ARM命令のRBIT+CLZに変換されます。

[4] ― 第4章のSIMDフリップ処理。各候補手に対してフリップマスクを計算します。

[5] ― 現在の盤面構造体を128ビットレジスタへ一括ロード。黒白2ビットボードを1命令で取り込みます。

[6] ― **NEON EOR命令で現局面とフリップマスクのXORを取り次局面を1命令で生成。黒白2ボードを同時処理します。**

[7] ― 128ビット一括ストアで次局面を構造体に書き出します。LD1→EOR→ST1の3命令で次局面生成が完結します。

◆ API仕様 / 引数説明

関数名 (intrinsic)	戻り値型	主な引数	引数・動作の説明
`__builtin_ctzll`	`int`	`unsigned long long x`	GCC/Clang組み込み関数。xの最下位から連続する0ット数(Count Trailing Zeros)を返す。ARM命令のRBIT+CLZに変換され1命令相当で実行。合法手マスから候補座標を取り出すのに使用。
`vld1q_u64`	`uint64x2_t`	`const uint64_t *ptr`	現在の盤面構造体を128bitレジスタへ一括ロード。2ビットボードを1命令で取り込みEOR命令への入力する。
`veorq_u64`	`uint64x2_t`	`uint64x2_t a, uint64x2_t b`	128bit XOR。現局面とフリップマスクのXORを取り局面を1命令で生成。黒白2ボードを同時処理し2回の別演算を1回に削減する。
`vst1q_u64`	`void`	`uint64_t *ptr, uint64x2_t val`	128bit一括ストアで次局面を構造体に書き出す。LD1→EOR→ST1の3命令だけで次局面生成が完結する。

10. まとめ

本資料では、NVIDIA DGX SparkのARM CPUに搭載されたNEON/SVE2 SIMD命令が、オセロ思考ルーチンの各処理段階でどのように活用できるかを、C++サンプルコード・SIMD命令解説・API仕様とともに解説しました。

活用領域	SIMD技法	主な利点
ビットボード表現	128bitレジスタに黒白格納	盤面全体を1レジスタで操作
合法手生成	8レーン並列シフト＋AND	8方向を1命令で同時探索
反転処理	可変シフト＋マスクAND連鎖	最頻出処理を大幅高速化
述語フィルタリング	述語レジスタ＋COMPACT	分岐予測ミスを完全排除
評価関数	MLA積和＋ADDVリダクション	内積計算を数命令に圧縮
石数カウント	CNTポピュレーションカウント	1命令でbitcount完了

DGX SparkのCortex-X925は1サイクルに6つのSIMD命令をリタイアできる実行幅を持ち、加えて128GBの統一メモリによるCPU-GPU間のゼロコピーデータ共有が可能です。

今後、SVE2の述語駆動型プログラミングとGPUのTensor Coreを組み合わせた**ハイブリッドAI探索**が、オセロに限らず様々なボードゲームAIの新しい標準となることが期待されます。

本資料に掲載したC++コードはarm_neon.hおよびarm_sve.hのintrinsicを用いた概念実装です。DGX Spark上でのコンパイルには、GCC/Clangで-march=armv8-a+sve2フラグを指定してください。